Maze2D Minimal Reference Note

Dian Kruger

Purpose

This note summarises the main technical facts established in the minimal Maze2D notebook, `maze_2d_minimal_plot.ipynb`. The goal is not to reproduce all code details, but to keep a concise record of how Maze2D is structured in D4RL/Minari and what this implies for a generative trajectory model.

Why Maze2D Was Studied

The longer-term thesis goal is to use a generative model to produce paths similar to those produced by reinforcement learning in Maze2D. Before designing such a model, it was necessary to understand:

- how mazes are defined,
- how episodes are stored,
- what observations, actions, positions, and velocities mean,
- how the MuJoCo physics update state,
- and how the reward is defined.

The minimal notebook was created for exactly this purpose: to expose the data structure and environment mechanics with as little extra code as possible.

Maze Definition

Maze layouts in D4RL are stored as compact strings. The notebook copies the relevant maze specifications directly from D4RL's `maze_model.py`.

Each maze is represented row by row using characters such as:

- `#`: wall cell,
- `O`: open cell,
- `G`: goal-designated cell in the layout definition.

In the notebook, a layout string is split into rows and then converted into a list of wall-cell coordinates. Those wall cells are drawn as black rectangles for plotting.

An important practical point is that the plotting frame must match the dataset frame. For the notebook plots, the wall cells are mapped into the dataset coordinate frame rather than plotted in raw maze-string row/column order.

Dataset Source and Structure

The notebook reads a local Minari dataset rather than relying on a standalone HDF5 file. In the current example, the dataset identifier is:

`analysis/maze2d/medium-raw-v0`

One episode contains a sequence of states, actions, and simulator information over time. For the examined Maze2D episode, the important arrays are:

- `episode.observations`: shape $(T, 4)$,
- `episode.actions`: shape $(T, 2)$,
- `episode.infos["goal"]`: shape $(T, 2)$,
- `episode.infos["qpos"]`: shape $(T, 2)$,
- `episode.infos["qvel"]`: shape $(T, 2)$.

The notebook also displays a table for the first few timesteps so that these fields can be inspected directly.

Observation, Position, and Velocity

In Maze2D, the observation is a four-dimensional vector:

$$\texttt{observation} = [x,\ y,\ v_x,\ v_y].$$

The simulator state is also stored more explicitly:

- `qpos` contains the MuJoCo position state, here $[x, y]$,
- `qvel` contains the MuJoCo velocity state, here $[v_x, v_y]$.

So in this environment:

$$\texttt{observation}[0:2] \approx \texttt{qpos}, \qquad \texttt{observation}[2:4] \approx \texttt{qvel}.$$

This distinction is useful conceptually:

- `observations` are the learning-facing state vectors,
- `qpos` and `qvel` are the low-level simulator state used for exact replay and debugging.

Actions

Each action is two-dimensional:

$$\texttt{action} = [a_x,\ a_y].$$

These actions are not endpoint coordinates. They are control inputs passed to the environment at each timestep. In the plots, arrows are drawn with `matplotlib.quiver`:

- the arrow base is the current position,
- the arrow direction comes from the stored action vector.

To keep the full-trajectory plot readable, only every `STRIDE`-th action is plotted.

Physics and Exact Replay

The notebook uses MuJoCo directly through D4RL's `MazeEnv`, which is defined in `maze_model.py`.

For exact replay, the notebook:

1. creates a `MazeEnv` with the chosen maze specification,
2. sets the initial MuJoCo state with `env.set_state(initial_qpos, initial_qvel)`,
3. steps through stored actions using `env.step(action)`,
4. records `env.sim.data.qpos` and `env.sim.data.qvel` after each step.

This mirrors the D4RL approach.

One implementation detail was needed on Windows: a writable local copy of `mujoco_py` is used inside the project because the package directory inside the virtual environment was permission-restricted for generated build files.

Reward Design

The reward definition comes from the environment code, not from the stored dataset rewards.

In `maze_model.py`, the `step()` function defines:

- sparse reward:

$$r = \begin{cases} 1 & \text{if } \|(x, y) - g\| \leq 0.5, \\ 0 & \text{otherwise,} \end{cases}$$

- dense reward:

$$r = \exp(-\|(x, y) - g\|).$$

So the dense version rewards proximity to the goal at every timestep, while the sparse version rewards entering a goal region.

However, the stored dataset rewards in `generate_maze2d_datasets.py` are written as zeros during dataset construction. Therefore, if one wants to understand the intended task objective, the environment code is the relevant source.

Cumulative Reward vs Greedy Reward

The reward is designed per step, but reinforcement learning policies are generally trained to maximise cumulative discounted return, not merely the immediate reward of the next action.

This matters in mazes. A purely greedy one-step rule could attempt to move directly toward the goal and become trapped against a wall. Cumulative return allows a policy to prefer action sequences that may temporarily move sideways or away from the goal if they eventually yield higher long-term reward by reaching the goal area.

That said, Maze2D's dense reward does not by itself encode an explicit step penalty such as -1 per timestep. So "reach the goal as fast as possible" is not written directly into the reward formula; it is encouraged indirectly through finite horizons and discounted return.

What the Minimal Notebook Shows

The notebook produces three kinds of useful diagnostic views:

- a full-episode path over the maze geometry,
- stored action arrows drawn along that path,
- a zoomed-in local replay over a few timesteps showing: current state, next state after applying the stored action, current velocity, and next velocity.

These plots help separate three ideas that are easy to conflate:

- where the agent is,
- what control input is applied,
- and how MuJoCo updates position and velocity in response.

Implications for a Generative Thesis Model

For a generative trajectory model, the main takeaway is that Maze2D trajectories are naturally sequential data. At each timestep there is a structured relation between:

- current state (x, y, v_x, v_y),
- action (a_x, a_y),
- next state,
- and a fixed or slowly varying goal.

This suggests several possible modelling choices:

- model only positions if the aim is geometric path generation,
- model state–action pairs if the aim is to stay close to environment dynamics,
- or model full state transitions if the aim is to generate physically plausible trajectories under control inputs.

The notebook supports the following conclusion: Maze2D is not just a shortest-path graph problem. It is a continuous control problem with walls, velocities, actions, MuJoCo dynamics, and a reward defined relative to a goal. Any generative replacement for RL should therefore be explicit about which level it is modelling:

- path geometry only,
- policy-like action sequences,
- or full dynamical trajectories.

Practical Summary

The most important facts to remember are:

1. Maze layouts are encoded as simple strings and converted into wall cells for plotting.
2. A Maze2D observation is $[x, y, v_x, v_y]$.
3. `qpos` is simulator position and `qvel` is simulator velocity.
4. Actions are 2D control inputs, not target coordinates.
5. Exact replay can be done by restoring `qpos`/`qvel` and stepping `MazeEnv` with stored actions.
6. The environment reward is defined in `maze_model.py`, not in the dataset rewards.
7. Dense reward encourages closeness to the goal; sparse reward only rewards entering the goal region.
8. RL optimises cumulative return, so wall avoidance must come from planning over future reward rather than greedy one-step motion.